

Colegio Técnico Profesional Vázquez de Coronado.

Especialidad:
Inteligencia Artificial

Subárea:
Programación

Título:
Proyecto I Semestre
Programación para visión por computadora

Profesora:
Karen Ugalde Reyes

Nombre del estudiante:
María Paula Obregón Calderón.

Sección:
12-3 B

Fecha de entrega: Miércoles 23 de abril, 2025

Tabla de Contenido:

Introducción:.....	2
Objetivo General:.....	3
Objetivos específicos:.....	3
Problemática:.....	3
Idea de programa:.....	4
Explicación uso de librerías:.....	4
Errores frecuentes y soluciones encontradas:.....	5
Problemas de bugs en el desarrollo de los programas:.....	5
Explicación de lo que más dificultad causó en el desarrollo del programa:.....	7
¿Se logró el motivo? Justifique su respuesta:.....	7
Evidencias:.....	8
Bibliografía:.....	10

Introducción:

En el mundo, existen varias problemáticas de las cuales se tiene solución, más solo lo ignoran, o simplemente, no saben como crearlo o implementarlo; como la comunicación, la violencia, la ausencia en lugares, entre muchas otras cosas. La visión por computadora se trata de un campo informático en donde se permite que las computadoras identifiquen y puedan comprender los objetos y personas que aparezcan en fotos o vídeos, también en donde se utiliza deep learning y redes neuronales convolucionales (CNNs).

Algunas ventajas de la visión por computadora sería la disponibilidad 24/7, no se cansa, no se queja, y su visión se mantiene igual en todo momento, Poseen mayor velocidad y precisión para realizar inspecciones más detallada, o adaptar su mira, más que un humano. Tienen múltiples aplicaciones, aunque depende mucho del ámbito en el cuál se requiera la visión por computadora. Por ejemplo, identificar defectos en productos, mejorar selfies con filtros, incluso desbloquear smartphones mediante reconocimiento facial.

Con el contexto proporcionado, la visión por computadora emerge como una herramienta poderosa capaz de resolver una amplia gama de problemas reales. A través de su capacidad para analizar imágenes y videos en tiempo real, es posible crear soluciones innovadoras en diversas áreas, como la seguridad, la salud, el medio ambiente, y la industria, entre otras. Esta tecnología no solo mejora la eficiencia y precisión en las tareas automatizadas, sino que también tiene el potencial de transformar áreas como el monitoreo de la salud, la prevención de accidentes, la optimización de procesos industriales y la mejora de la calidad de vida.

A continuación, se explicará la problemática planteada por mi persona, los objetivos específicos y general, además y la idea del programa para abordar esta problemática.

Objetivo General:

- Diseñar e implementar un sistema de detección de fatiga en conductores mediante visión por computadora, en donde analice señales faciales en tiempo real a partir de videos en vivo o pregrabados, con la capacidad de emitir una alerta automática al detectar signos de somnolencia.

Objetivos específicos:

1. Planear y desarrollar un sistema de alerta para conductores por medio de la visión por computadora, que sea capaz de detectar signos de fatiga ocular en el conductor mediante el cierre prolongado de ojos en secuencias de video, ya sea en vivo o uno pregrabado.
2. Implementar un algoritmo de detección de rostros y puntos faciales, con la ayuda de python y de librerías como OpenCV, Numpy, MediaPipe y Pygame desde el bash de anaconda powershell o el cmd de windows (preferiblemente el bash de anaconda powershell D:)
3. Alertar al conductor con el fin de prevenir accidentes automovilísticos.

Problemática:

- Todos sabemos que las jornadas laborales/estudiantiles son exhaustivas, algunas veces hasta llegamos al punto de dormir tarde o incluso trasnochamos completamente para después no dormir nada durante toda la noche solamente por estar haciendo deberes que no tuvimos tiempo de terminar para entregarlo al día siguiente; aunque el insomnio también cuenta, en el caso de los conductores, se supone que tienen que tener reflejos y estar atentos a cualquier situación que aparezca de la nada en la calle, uno de los lugares más peligrosos e improbables si es que no sabes manejar bien del todo. Cuáles serían las consecuencias de esta causa? Bueno, podría causar lentitud a la reacción, choques, atropellos, e incluso la muerte de alguien en el proceso, ya sea del mismo conductor, de los pasajeros, o de las personas que estén en la vía ya sea cruzando la calle o incluso manejando otro vehículo.

Idea de programa:

- Con esta problemática en mente, mi idea para resolver (o manejar un poco la situación en la carretera con los conductores) es un detector de fatiga, donde el programa analizará las acciones de los ojos del conductor a cargo del vehículo para alertar y despertar un poco al mismo con una alarma integrada, esto para seguir en el camino de manera exitosa y sin distracciones.

Explicación uso de librerías:

- A continuación, las librerías que se utilizaron fueron las siguientes:

Librería:	Función:	Contexto en el código:
cv2 (OpenCV) versión: 4.5.5.64	Procesa imágenes y videos	Se utilizó para capturar video ya sea desde una cámara o un archivo (VideoCapture()), para mostrar formas (cómo los primeros intentos del borrador que tenía) y también para poner texto (con cv2.putText() para mostrar: "Alerta de fatiga" en la pantalla)
MediaPipe Versión: 0.10.5	Funciona para detección facial avanzada.	Encuentra los ojos y los rasgos del rostro fácilmente (esto como alternativa de dlib ya que tuve un problema el cual explicaré más adelante)
Numpy Versión: 1.23.5	Se utiliza para calcular números con vectores/matrices.	Se calcula el EAR (Eye Aspect Ratio), en donde mide la relación entre la apertura del ojo, para ver si los ojos se mantienen cerrados por algunos frames consecutivos
Pygame Versión: 2.1.0	Crea aplicaciones multimedia.	Encargado de agregar sonido, se utilizó para agregar una alarma cuando se detectan acciones de fatiga.

Errores frecuentes y soluciones encontradas:

1. El primer problema que tuve al hacer el programa, fue instalar las librerías, ya que lo estaba haciendo desde anaconda navigator, consulté primero en páginas para verificar cuales librerías eran las mejores para utilizar, y me di cuenta que si lo hacía desde el navegador, me tardaría más buscando librería por librería en orden alfanumérico, así que creé un entorno nuevo llamado “proyecto” (en Anaconda Navigator), lo activé y descargué las librerías desde el bash de anaconda (Anaconda Powershell Prompt) para finalmente utilizarlo en Visual Studio Code, el cuál elegí ya que es el programa con el cuál me siento más cómoda para programar.
2. El segundo problema llegó cuando tenía que descargar la librería “dlib”, y tuve que hacer muchos pasos para que al final tarde 2 horas sin hacer nada por querer instalarse, después de la tan larga espera, dejé de intentar con dlib y traté de buscar otra librería, en donde decidí trabajar con MediaPipe (que es propiedad de Google). Y me fue mucho mejor de lo que tenía en mente.
3. El tercer y último problema que tuve fue que el programa ahora no lograba ejecutarse en la máquina virtual, curiosamente el Domingo 06-04-2024 funcionaba a la perfección en mi casa; Tal vez puede ser un problema de espacio (Ya que la máquina virtual solo tenía 50GB y no 500GB como tenía que ser), incluso la ausencia del prompt de python el cual puede no sea acorde a la versión, además, MediaPipe dejó de funcionar normalmente. Traté de ejecutar el programa y bueno, decía que el módulo no existía. El lunes 07-04-2025 hice una máquina virtual desde cero, ahora con 500GB de almacenamiento para que no suceda de nuevo. Y el miércoles 09-04-2025 pude arreglar este problema, al hacer un entorno nuevo en anaconda navigator llamado “Detector” con Python 3.9, y descargué todas las librerías necesarias en ese entorno con la versión correspondiente para que fueran compatibles con Python, el problema sucedió ya que algunas versiones no tenían compatibilidad entre sí. Pero cuando descargué todas las librerías con su correspondiente versión. Funcionó otra vez de maravilla.

Problemas de bugs en el desarrollo de los programas:

- Problemas de descargar dlib y que no quería bajarse al powershell: Bueno, dlib funciona para el reconocimiento facial, y como dije anteriormente, esto me dió un grave problema que retrasó mi proceso por esperar 2 horas a que se instale, necesitaba muchos archivos, los datos estaban compilados en C++ y mientras estaba tratando de instalarlo desde el bash, perdí tiempo valioso, así que por eso pasé a MediaPipe, que hace algo parecido con el reconocimiento facial.
- Cuadros que miraban a la nariz y no a los ojos: En el primer intento de código para el detector, implementé unos cuadros para que se queden en los ojos y verificar si la persona estaba parpadeando o si tenía los ojos cerrados por unos instantes para activar la alarma. Lo cual, no fue muy efectivo que digamos, en realidad los cuadros apuntaban a los orificios de la nariz, hasta apuntaban a paredes y cuadros atrás en los videos de prueba, por lo tanto, intenté con algo mejor calibrado, entonces se hizo con matrices en forma de puntos alrededor de los ojos para dar más visibilidad a la acción de los ojos del conductor. Si parpadea, no pasará nada, pero si cierra los ojos por unos breves momentos, la alarma sonará.

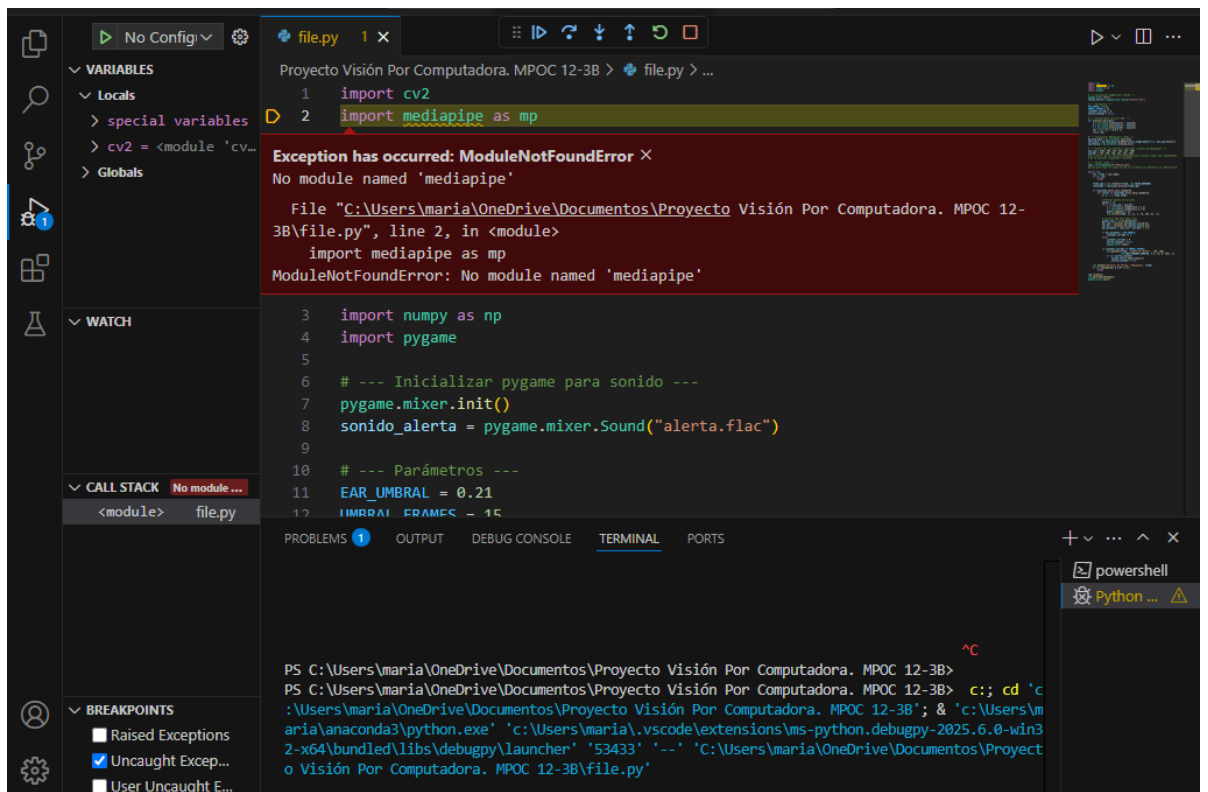
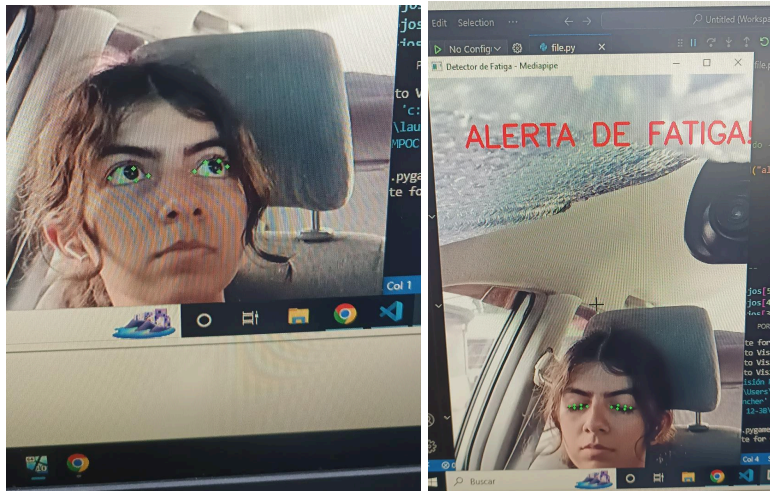
Explicación de lo que más dificultad causó en el desarrollo del programa:

- Descargar las librerías desde el bash de anaconda powershell y el cmd de windows, esto ya que cuando descargaba una librería, borraba otra, ya después entendí que borraba el que tenía una versión más vieja y se actualizaba a la más nueva o necesitada, como dije anteriormente, tuve que ver muchas alternativas al ver que dlib tardaba mucho en descargarse, y cuando MediaPipe ya no funcionaba del todo en la máquina virtual, pasé 3 días tratando de encontrar una solución, para finalmente hacer todo desde cero, descargar las librerías adecuadas, configurar python desde el entorno de anaconda para que las librerías pudieran trabajar, y verificar que el programa pudiera ejecutarse con el entorno adecuado.

¿Se logró el motivo? Justifique su respuesta:

- Diría que sí, lastimosamente con la falta de webcam como tal, tuve que recurrir a la importación de vídeo para poder hacer pruebas como tal, las cuales funcionaron exitosamente, habían problemas de diseño, matrices e incluso de sonido, pero al final sí se logró hacer el detector, tal vez no de una manera tan elaborada, pero podría decirse que es un “leve borrador” para crear una idea más y más grande.

Evidencias:



Home

Environments

Learning

Community

Anaconda Assistant

Documentation

Anaconda Blog

Search Environments

base (root)

Detector

Proyecto

Create Clone Import Backup Remove

Installed Channels Update index... Search Packages

Name	T	Description	Version
matplotlib		Publication quality figures in python	3.9.4
mediapipe			0.10.5
numpy		Array processing for numbers, strings, records, and objects.	1.23.5
opencv-contrib-python			4.11.0.8
opencv-python			4.5.5.64
openssl		Openssl is an open-source implementation of the ssl and tls protocols	3.0.16
packaging		Core utilities for python packages	24.2
pillow		Pillow is the friendly pil fork by alex clark and contributors	11.1.0
pip		Pypa recommended tool for installing python packages	25.0

34 packages available

```

Anaconda Powershell Prompt
Requirement already satisfied: pyjssparser>=2.5.1 in c:\users\maria\anaconda3\lib\site-packages (from js2py->pipwin) (2.7.1)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in c:\users\maria\anaconda3\lib\site-packages (from requests->pipwin) (1.26.14)
Requirement already satisfied: charset-normalizer<3,>=2 in c:\users\maria\anaconda3\lib\site-packages (from requests->pipwin) (2.0.4)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\maria\anaconda3\lib\site-packages (from requests->pipwin) (2022.12.7)
Requirement already satisfied: idna<4,>=2.5 in c:\users\maria\anaconda3\lib\site-packages (from requests->pipwin) (3.4)
Requirement already satisfied: tzdata in c:\users\maria\anaconda3\lib\site-packages (from tzlocal>=1.2->js2py->pipwin) (2025.2)
(base) PS C:\Users\maria> conda install -c conda-forge dlib
Collecting package metadata (current_repodata.json): done
Solving environment: failed with initial frozen solve. Retrying with flexible solve.
Solving environment: -
  
```

Bibliografía:

Microsoft. (s.f). *¿Qué es la visión artificial?*.

<https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-is-computer-vision#:~:text=Las%20aplicaciones%20de%20visi%C3%B3n%20artificial,el%20contenido%20de%20otras%20im%C3%A1genes>

IBM. (s.f). *¿Qué es la visión por computadora?*.

<https://www.ibm.com/es-es/topics/computer-vision>

Zapater, S. (2022). *¿Qué es la visión artificial por computador?*.

<https://www.hiberus.com/crecemos-contigo/vision-artificial-por-computador/#:~:text=Alimentaci%C3%B3n,las%20empresas%20de%20este%20sector>

Cloud Devs. (s.f). *10 Python Libraries for Computer Vision*.

<https://clouddevs.com/python/libraries-for-computer-vision/#:~:text=OpenCV%20is%20the%20go%2Dto,%2C%20object%20detection%2C%20and%20more>

Zyprian, F. (2023). *OpenCV - Guía cv2 para desarrolladores de Python*.

<https://konfuzio.com/es/cv2/>

Nerd Paradise. (s.f). *PyGame Tutorial: Music and Sound Effects*.

<https://nerdparadise.com/programming/pygame/part3>

Google. (2025). *Guía de soluciones de MediaPipe*.

<https://ai.google.dev/edge/mediapipe/solutions/guide?hl=es-419>